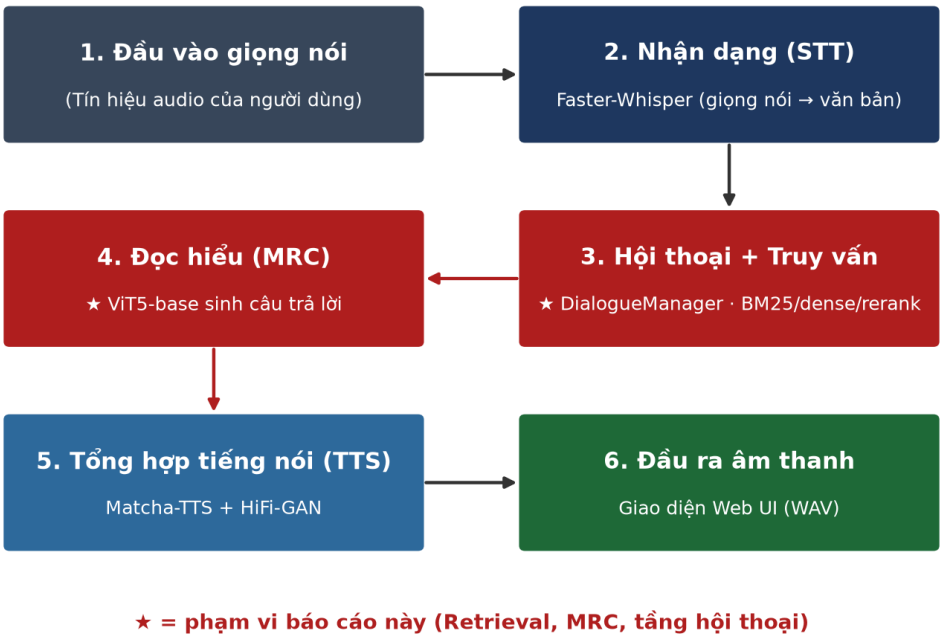
CHƯƠNG 1: GIỚI THIỆU

1.1 BỐI CẢNH VÀ VAI TRÒ TRONG HỆ THỐNG

Trong hệ thống hỏi đáp đọc hiểu văn bản bằng giọng nói tiếng Việt (*Vietnamese Multiple-choice Machine Reading Comprehension Voice Q&A* - ViMMRC Voice Q&A), phần việc của tác giả gồm ba khâu nằm giữa nhận dạng giọng nói và tổng hợp tiếng nói: **truy xuất ngữ cảnh** (*Retrieval*), **đọc hiểu máy** (*Machine Reading Comprehension* - MRC) và **tầng hội thoại** (*Dialogue Manager*). Đầu vào của chuỗi này là câu hỏi dạng văn bản do module nhận dạng giọng nói (STT) tạo ra; đầu ra là một câu trả lời hoàn chỉnh để module Text-to-Speech đọc lên.

Về mặt kỹ thuật, bài toán được chia thành ba việc nối tiếp. Thứ nhất, từ một câu hỏi tự do, tìm đoạn văn có khả năng chứa đáp án trong kho tri thức hơn 1,48 triệu đoạn. Thứ hai, từ cặp (câu hỏi, đoạn văn), mô hình ViT5 sinh ra đáp án dạng văn bản ngắn. Thứ ba, tầng hội thoại xử lý những câu nói không phải câu hỏi tra cứu đơn (xã giao, câu nối tiếp, so sánh, đúng/sai) và soạn đáp án thành câu trọn vẹn.



Hình 1.1: Vị trí của Retrieval, MRC và tầng hội thoại trong chuỗi xử lý của hệ thống ViMMRC Voice Q&A

Nguồn: Tác giả tự thiết kế từ kiến trúc hệ thống (2026)

1.2 MỤC TIÊU NGHIÊN CỨU VÀ PHÁT TRIỂN

Phần việc đặt ra các mục tiêu sau, được đo bằng các chỉ số định lượng ở Chương 6:

- **Truy xuất đúng đoạn văn:** xếp đoạn chứa đáp án lên đầu danh sách (Recall@1, Recall@5), cả với kho nhỏ đã biết lẫn kho Wikipedia quy mô triệu đoạn.
- **Đọc hiểu sinh đáp án tự do:** mô hình sinh (*generative*) trả lời bằng văn bản, không phụ thuộc phương án trắc nghiệm, đo bằng Exact Match (EM) và token F1.
- **Chạy được trên CPU thông thường:** độ trễ của khâu truy xuất và đọc hiểu đủ thấp để hệ thống phản hồi trong khoảng 6 đến 12 giây mỗi câu hỏi khi chạy Docker trên CPU.
- **Hội thoại tự nhiên:** hiểu câu hỏi nối tiếp theo phiên, từ chối khi kho không có thông tin thay vì bịa đáp án, và luôn trả lời bằng câu hoàn chỉnh cho giọng đọc.

1.3 CÁC ĐÓNG GÓP CHÍNH CỦA ĐỀ TÀI

Phần việc mang lại năm đóng góp kỹ thuật chính:

- **Thứ nhất:** xây dựng bộ truy xuất BM25 tự cài đặt, chọn cấu hình bằng câu hỏi tập huấn luyện (16 cấu hình ứng viên) và kiểm chứng một lần trên tập dev: Recall@1 tăng từ 71,45% lên 77,48% trên 564 câu hỏi.
- **Thứ hai:** dựng kho tri thức hai tầng: kho chọn lọc 5.879 đoạn và 1,48 triệu đoạn Wikipedia tiếng Việt, kết hợp BM25, truy xuất vector (multilingual-e5-small) và bộ xếp hạng lại bge-reranker-v2-m3.
- **Thứ ba:** mở rộng dữ liệu đọc hiểu bằng cách gộp ViMMRC 2.0 với UIT-ViQuAD2.0 thành 27.164 câu hỏi, chia 80/10/10 theo đơn vị đoạn văn để tránh rò rỉ giữa các tập.
- **Thứ tư:** tinh chỉnh ViT5-base theo hai giai đoạn, đạt EM 41,05% và token F1 66,55% trên tập test hỗn hợp 2.716 câu; đồng thời ghi nhận kết quả âm (fine-tune riêng ViMMRC, thay mô hình bằng LLM 9B) để khẳng định giới hạn của bài toán.
- **Thứ năm:** xây tầng hội thoại bằng quy tắc và heuristic, không thêm thư viện, đạt 59/72 mục (81,9%) ở lần chạy đầu trên bộ held-out tự viết.

CHƯƠNG 2: THIẾT KẾ GIẢI PHÁP TỔNG THỂ

2.1 QUY CÁCH DỮ LIỆU ĐẦU VÀO VÀ ĐẦU RA

2.1.1 Hợp đồng giao diện giữa các module

Mọi module trong hệ thống trao đổi bằng văn bản thuần hoặc đường dẫn âm thanh, nên ba khâu của tác giả được thiết kế với giao diện Python nhỏ gọn như Bảng 2.1. Retriever và reader được khởi tạo một lần khi ứng dụng khởi động, sau đó dùng lại cho mọi yêu cầu.

Bảng 2.1: Hợp đồng giao diện của các module Retrieval, MRC và hội thoại

Module	Hàm chính	Đầu vào	Đầu ra
Retrieval	Retriever.search(q, top_k)	Câu hỏi (str), số kết quả	Danh sách {id, passage, score}
Retrieval	Retriever.retrieve(q)	Câu hỏi (str)	Đoạn văn tốt nhất (str, rỗng nếu không khớp từ nào)
MRC	MRCClient.answer_question(q, passage)	Câu hỏi và đoạn văn (str)	Đáp án (str)
Hội thoại	DialogueManager.respond(text, session_id, passage)	Câu nói, mã phiên, đoạn văn tùy chọn	{answer, kind, resolved_question, passage, ...}

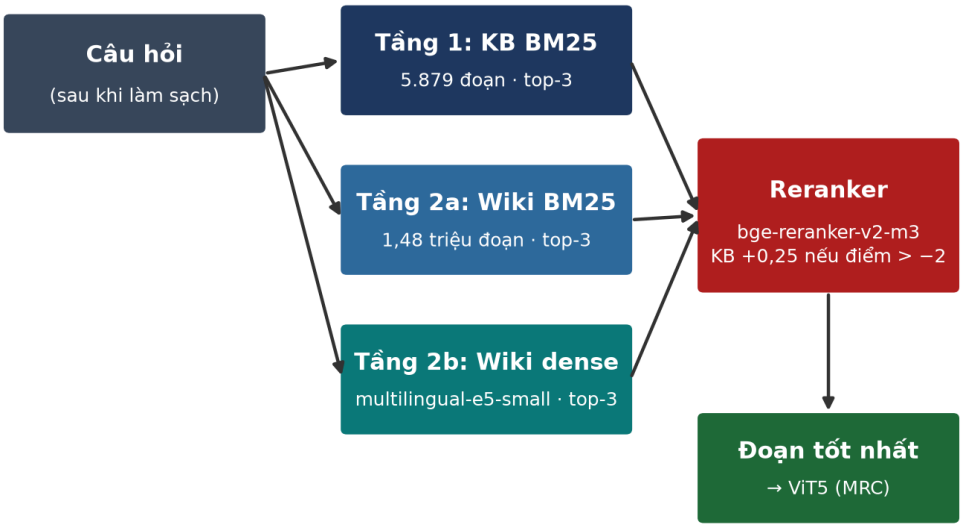
Nguồn: Tác giả tổng hợp từ mã nguồn các module retrieval-model_Huy, mrc-model_Huy, dialogue-manager_Huy (2026)

2.1.2 Giao thức tích hợp hệ thống

Các khâu được gắn vào máy chủ FastAPI của dự án. Web UI gọi một route duy nhất /api/ask (nhận question, session_id, passage tùy chọn và trả answer, kind, resolved_question, passage); các route /api/retrieve và /api/answer vẫn được giữ để kiểm thử từng khâu riêng lẻ. Khi chưa có dữ liệu hoặc checkpoint, hệ thống tự lùi về kho mẫu 75 đoạn và bộ đọc heuristic để vẫn chạy được ngoại tuyến.

2.2 LUỒNG XỬ LÝ DỮ LIỆU TRONG HỆ THỐNG

Một câu nói đi qua các giai đoạn trong Bảng 2.2. Điểm khác với một hệ thống RAG thông thường là bước truy xuất không chỉ lấy một danh sách từ một chỉ mục, mà gom ứng viên từ ba nguồn rồi để bộ xếp hạng lại chọn chung, như Hình 2.1.



Hình 2.1: Luồng truy xuất ngữ cảnh hai tầng kết hợp BM25, truy xuất vector và bộ xếp hạng lại

Nguồn: Tác giả thiết kế từ cấu hình TieredRetriever trong kb_retriever.py (2026)

Bảng 2.2: Các giai đoạn xử lý của một lượt hỏi đáp

Giai đoạn	Nội dung kỹ thuật thực hiện
1. Làm sạch câu nói	Bỏ từ đệm và phần bao ngoài ("ừm", "cho mình hỏi là", "... không"), chuẩn hóa khoảng trắng.
2. Định tuyến ý định	Nhận diện xã giao, phép tính, câu nối tiếp, so sánh, câu ghép, đúng/sai; câu hỏi đơn đi tiếp xuống truy xuất.
3. Truy xuất	Lấy top-3 từ BM25 kho chọn lọc, top-3 BM25 Wikipedia, top-3 vector Wikipedia.
4. Xếp hạng lại	bge-reranker-v2-m3 chấm chung các ứng viên; đoạn thuộc kho chọn lọc được cộng 0,25 khi điểm thô lớn hơn -2.
5. Đọc hiểu	ViT5 nhận cặp (câu hỏi, đoạn văn) và sinh đáp án ngắn; chọn đáp án bằng beam search.
6. Soạn câu trả lời	Biến cụm ngắn thành câu trọn vẹn, từ chối khi đoạn văn không chứa từ khóa hoặc tên riêng của câu hỏi.

Nguồn: Tác giả tổng hợp từ thiết kế hệ thống (2026)

CHƯƠNG 3: CƠ SỞ LÝ THUYẾT

3.1 TRUY XUẤT TỪ KHÓA BẰNG BM25

BM25 (Robertson và Zaragoza, 2009) là hàm xếp hạng dựa trên tần suất từ, xuất hiện trong hầu hết hệ truy xuất văn bản hiện đại như một đường cơ sở mạnh. Với câu hỏi q và đoạn văn d, điểm số được tính theo công thức (3.1), trong đó $f(t,d)$ là số lần từ t xuất hiện trong d, |d| là độ dài đoạn, avgdl là độ dài trung bình của kho, k_1 điều khiển độ bão hòa tần suất và b điều khiển mức chuẩn hóa theo độ dài.

$$\text{score}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\,(1 - b + b\,|d|/\text{avgdl})} \qquad (3.1)$$

Trọng số IDF của từ phụ thuộc số đoạn n_t chứa từ đó trên tổng số N đoạn như công thức (3.2). Từ càng hiếm thì càng đóng góp nhiều vào điểm số.

$$\text{IDF}(t) = \ln\left(1 + \frac{N - n_t + 0{,}5}{n_t + 0{,}5}\right) \qquad (3.2)$$

Tokenizer của đề tài chuẩn hóa Unicode NFC, tách từ bằng biểu thức chính quy theo âm tiết (không tách từ ghép tiếng Việt) và không xử lý đồng nghĩa. Vì BM25 chỉ so khớp bề mặt, câu hỏi diễn đạt khác đoạn văn sẽ bị truy xuất kém; đây là lý do hệ thống bổ sung truy xuất vector và bộ xếp hạng lại.

3.2 TRUY XUẤT VECTOR VÀ XẾP HẠNG LẠI

Truy xuất vector (*dense retrieval*; Karpukhin và cộng sự, 2020) mã hóa câu hỏi và đoạn văn thành vector rồi so khớp bằng tích vô hướng. Đề tài dùng multilingual-e5-small (Wang và cộng sự, 2024) với vector 384 chiều, lưu dạng float16 (khoảng 1,1 GB cho 1,48 triệu đoạn) và thêm tiền tố "query: " cho câu hỏi, "passage: " cho đoạn văn đúng như mô hình yêu cầu.

Bộ xếp hạng lại (*cross-encoder reranker*; Nogueira và Cho, 2019) đọc đồng thời câu hỏi và từng ứng viên để chấm điểm liên quan, chính xác hơn so khớp độc lập nhưng chậm hơn nhiều, nên chỉ áp dụng cho một nhóm nhỏ ứng viên. Đề tài dùng bge-reranker-v2-m3 (Chen và cộng sự, 2024).

3.3 ĐỌC HIỂU DẠNG SINH VỚI VIT5

ViT5 (Phan và cộng sự, 2022) là mô hình text-to-text kiến trúc encoder-decoder theo T5 (Raffel và cộng sự, 2020), tiền huấn luyện trên ngữ liệu tiếng Việt. Thay vì dự đoán vị trí đầu và cuối của đáp án trong đoạn văn, mô hình nhận chuỗi đầu vào "Trả lời câu hỏi dựa trên đoạn văn. Câu hỏi: ... Đoạn văn: ..." và sinh trực tiếp đáp án. Cách này dùng chung một mô hình cho cả dữ liệu trắc nghiệm lẫn tự luận.

CHƯƠNG 4: PHƯƠNG PHÁP TRIỂN KHAI VÀ TỐI ƯU KỸ THUẬT

4.1 TỐI ƯU BỘ TRUY XUẤT BM25

4.1.1 Chọn cấu hình bằng câu hỏi tập huấn luyện

Bộ truy xuất BM25 được tự cài đặt bằng chỉ mục đảo (*postings*), chỉ chấm điểm các đoạn chứa từ của câu hỏi. Cách này nhanh hơn 2,74 lần so với quét toàn bộ kho (0,0518 giây so với 0,1420 giây trên 200 câu hỏi) mà cho cùng top-5. Để tránh chọn tham số theo tập dev, script benchmark_retrieval.py lấy mẫu cố định 800 câu hỏi huấn luyện (seed 42), chấm 16 cấu hình theo thứ tự Recall@1, MRR@5, Recall@5, rồi mới đo cấu hình thắng một lần trên dev. Không đọc tập test, đáp án A-D hay phương án trả lời.

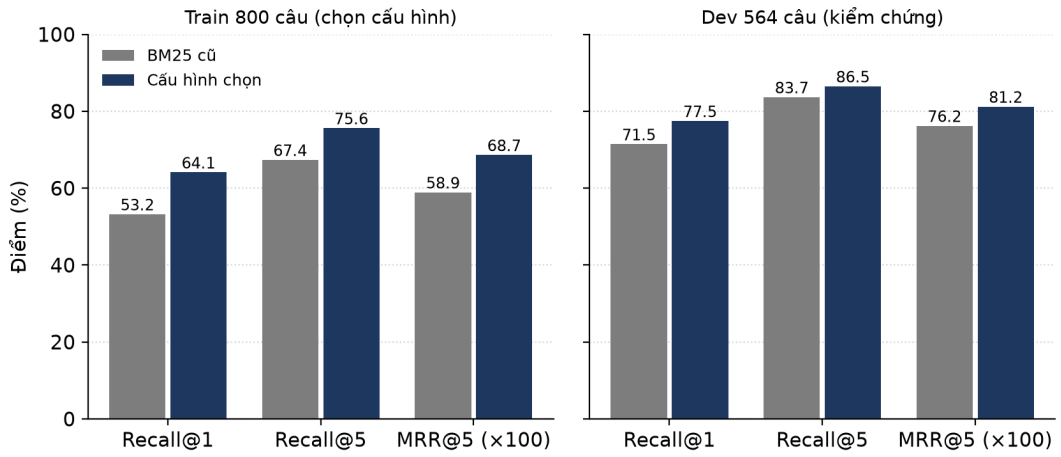
Bảng 4.1: So sánh cấu hình BM25 gốc và cấu hình được chọn

Tham số	**BM25 cũ**	**Cấu hình chọn**
k_1	1,5	2,1
b	0,75	1,0
Bỏ từ dừng của câu hỏi (query stopwords)	Không	Có (chỉ với unigram)
Hệ số thưởng bigram λ	0	0,4

Nguồn: Tác giả tổng hợp từ retrieval-model_Huy/reports/LEXICAL_BENCHMARK.md (2026)

Điểm bigram thưởng nhẹ các cặp từ đứng cạnh nhau trong câu hỏi và trong đoạn văn, theo công thức (3.3). Điều này giúp phân biệt các cụm như "Quang Trung" với những đoạn chỉ chứa từng từ riêng lẻ.

$$\text{score}'(q,d) = \text{score}_{\text{uni}}(q,d) + \lambda \cdot \text{score}_{\text{bi}}(q,d), \quad \lambda = 0{,}4 \qquad (3.3)$$



Hình 4.1: Hiệu quả truy xuất của BM25 gốc và cấu hình được chọn trên tập chọn cấu hình và tập dev

Nguồn: Tác giả tổng hợp từ retrieval-model_Huy/reports/lexical_benchmark.json (2026)

4.2 KHO TRI THỨC HAI TẦNG VÀ XẾP HẠNG LẠI

Kho dùng trong hệ thống chạy thật gồm hai tầng như Bảng 4.2. Tầng chọn lọc chứa các bài đọc có nguồn rõ ràng và nhỏ đủ để dùng BM25 có điểm bigram; tầng Wikipedia lớn hơn gần 250 lần nên chỉ dùng BM25 thư viện bm25s (không bigram, vì chỉ mục bigram cho hàng trăm nghìn đoạn không vừa bộ nhớ máy xách tay) cùng chỉ mục vector.

Bảng 4.2: Các tầng của kho tri thức

Tầng	Quy mô	Cách truy xuất
Tầng 1: kho chọn lọc	5.879 đoạn (552 ViMMRC, 5.317 ViQuAD, 10 thẻ tri thức)	BM25 tự cài đặt, k_1 2,1, b 1,0, thưởng bigram 0,4
Tầng 2: Wikipedia tiếng Việt	1,48 triệu đoạn, chia 4 shard	BM25 (bm25s) và vector multilingual-e5-small 384 chiều
Xếp hạng lại	Top-3 mỗi nguồn (tối đa 9 ứng viên)	bge-reranker-v2-m3; kho chọn lọc +0,25 nếu điểm thô > -2

Nguồn: Tác giả tổng hợp từ STATUS.md và retrieval-model_Huy/kb_retriever.py (2026)

Điểm BM25 của hai tầng không so sánh trực tiếp được vì khác chỉ mục. Phương án ban đầu là chọn tầng theo tỷ lệ điểm BM25 (ngưỡng 0,4, tinh chỉnh bằng scripts/tune_kb_threshold.py). Phương án hiện dùng đưa mọi ứng viên cho bộ xếp hạng lại chấm chung và cộng thưởng $\beta = 0{,}25$ cho tầng chọn lọc theo công thức (3.4). Thưởng chỉ áp dụng khi điểm thô lớn hơn -2: kho chọn lọc nhỏ, đoạn của nó dễ "thắng nhầm" với câu hỏi kiến thức mở, nên ngưỡng này ngăn một đoạn có điểm rất thấp được nâng hạng chỉ nhờ thưởng.

$$s_{\text{final}}(q,d) = r(q,d) + \beta \cdot \mathbb{1}[d \in \text{KB},\ r(q,d) > -2], \quad \beta = 0{,}25 \qquad (3.4)$$

4.3 DỮ LIỆU ĐỌC HIỂU HỢP NHẤT

ViMMRC 2.0 chỉ có 5.273 câu hỏi trắc nghiệm trích từ sách giáo khoa, nên mô hình sinh đáp án học được ít dạng câu hỏi. Script build_combined_dataset.py gộp thêm UIT-ViQuAD2.0 (công khai, câu hỏi tự luận trích xuất từ Wikipedia). Câu hỏi không có đáp án (*is_impossible*) bị loại: 28.454 câu gốc còn 21.891 câu. Toàn bộ pool được chia lại 80/10/10 theo **đơn vị đoạn văn** với seed 42, vì chia theo câu hỏi sẽ để cùng một đoạn xuất hiện ở cả train lẫn test và làm số liệu bị lạc quan.

Bảng 4.3: Thành phần bộ dữ liệu đọc hiểu hợp nhất

Thành phần	Số câu hỏi	Ghi chú
ViMMRC 2.0	5.273	Trắc nghiệm 4 đáp án; đáp án = văn bản phương án đúng
UIT-ViQuAD2.0 (đã lọc)	21.891	Tự luận; lấy từ 28.454 câu sau khi bỏ câu không có đáp án
Tổng	27.164	Chia theo đoạn văn, seed 42
Train / Dev / Test	21.733 / 2.715 / 2.716	80% / 10% / 10%

Nguồn: Tác giả tổng hợp từ mrc-model_Huy/README.md và build_combined_dataset.py (2026)

4.4 TẦNG HỘI THOẠI

Tầng hội thoại nằm giữa STT và truy xuất, chỉ dùng biểu thức chính quy và heuristic nên không tốn thêm mô hình. Mục tiêu là xử lý các câu nói mà một cặp (câu hỏi, đoạn văn) đơn lẻ không giải quyết được:

- **Làm sạch câu nói:** bỏ từ đệm của giọng nói ("ừm", "cho mình hỏi là") trước khi truy xuất.
- **Câu nối tiếp theo phiên:** viết lại "còn Hàn Quốc thì sao?" thành "Thủ đô của Hàn Quốc là gì" và thay đại từ "ông ấy" bằng thực thể ở lượt trước, dựa trên session_id.
- **Câu phức:** tách câu ghép thành hai câu hỏi, xử lý so sánh ("A hay B dài hơn") và đúng/sai bằng cách đối chiếu mệnh đề với đoạn bằng chứng (tên riêng, con số, phủ định).
- **Soạn câu trả lời:** đổi cụm ngắn của ViT5 như "Tokyo" thành "Thủ đô của Nhật Bản là Tokyo." và từ chối khi đoạn văn không chứa từ khóa hay tên riêng của câu hỏi, thay vì đưa đáp án không có căn cứ.

CHƯƠNG 5: QUY TRÌNH HUẤN LUYỆN MÔ HÌNH ĐỌC HIỂU

5.1 DỮ LIỆU VÀ ĐỊNH DẠNG HUẤN LUYỆN

- **Mô hình nền:** VietAI/vit5-base (khoảng 220 triệu tham số), tinh chỉnh toàn bộ trọng số.
- **Định dạng:** đầu vào "Trả lời câu hỏi dựa trên đoạn văn. Câu hỏi: ... Đoạn văn: ...", tối đa 512 token; đầu ra tối đa 64 token. Nhãn đệm bằng -100 nên phần padding không tham gia hàm mất mát.
- **Kiểm soát rò rỉ:** train.py xác nhận không có đoạn văn trùng giữa các tập trước khi nạp mô hình và báo số đầu vào bị cắt token.
- **Giới hạn đã biết:** đoạn dài hơn 512 token bị cắt phần cuối, nên thông tin nằm cuối bài đọc có thể mất.

5.2 BA GIAI ĐOẠN HUẤN LUYỆN

Quá trình huấn luyện thực hiện trên GPU với BF16 khi phần cứng hỗ trợ (script từ chối chạy huấn luyện đầy đủ trên CPU). Kích thước batch là 1 và tích lũy gradient 16 nên batch hiệu dụng là 16. Trainer chọn checkpoint theo Exact Match trên **dev**, không phải checkpoint cuối, rồi mới dự đoán **test** một lần.

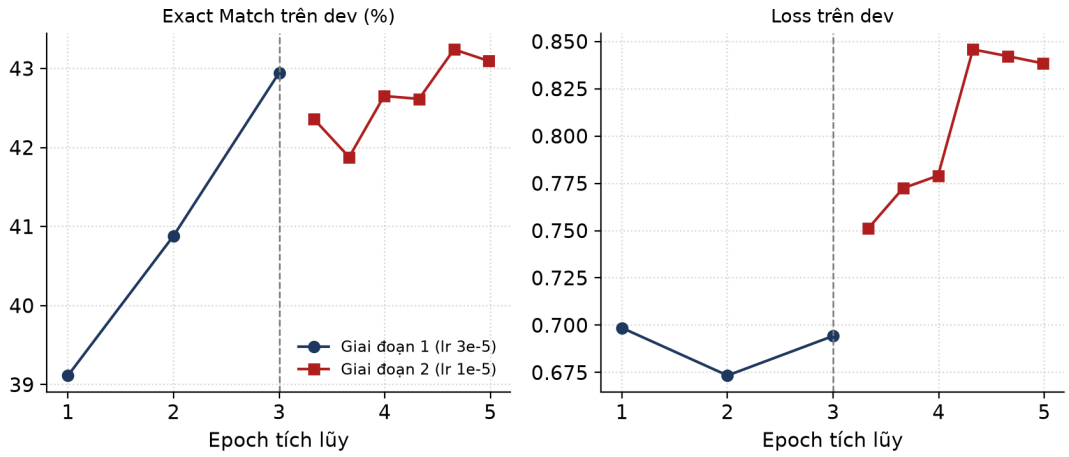
Bảng 5.1: Thiết lập ba giai đoạn huấn luyện mô hình ViT5

Giai đoạn	Khởi tạo từ	Số epoch	Learning rate	Dev EM tốt nhất
1. vit5-combined	VietAI/vit5-base	3	3e-5	42,95% (epoch 3)
2. vit5-combined-cont (đang dùng)	vit5-combined	2	1e-5	43,24% (bước 2250)
3. vit5-vimmrc-ft (thử nghiệm)	vit5-combined-cont	3	-	10,63% (epoch 2)

Nguồn: Tác giả trích từ trainer_state.json của ba checkpoint (2026). Dấu - = không ghi trong tệp trạng thái; giai đoạn 3 đo trên dev ViMMRC riêng nên không cùng thước đo với hai dòng trên

5.2.1 Giai đoạn 2 và dấu hiệu quá khớp

Giai đoạn 2 tiếp tục huấn luyện hai epoch với learning rate nhỏ hơn và lưu checkpoint mỗi 450 bước để khôi phục khi máy tắt đột ngột. Dev EM nhích từ 42,95% lên 43,24% nhưng dev loss tăng từ 0,67-0,69 lên 0,84 (Hình 5.1) trong khi train loss tiếp tục giảm, cho thấy mô hình bắt đầu quá khớp. Vì vậy tác giả dừng tại đây và không huấn luyện thêm với cùng cấu hình.



Hình 5.1: Exact Match và loss trên dev qua hai giai đoạn huấn luyện

Nguồn: Tác giả vẽ từ trainer_state.json của vit5-combined và vit5-combined-cont (2026)

5.2.2 Thử nghiệm fine-tune riêng ViMMRC

Để kiểm tra giả thuyết "dữ liệu miền hẹp giúp đúng miền đó", tác giả tinh chỉnh tiếp mô hình chỉ trên ViMMRC thuần. Dev EM dừng quanh 10% và dev loss tăng đều (2,18 → 2,36) qua ba epoch, nên huấn luyện dừng sớm. Trên toàn chuỗi (Bảng 6.3), mô hình này chỉ chênh vài phần mười điểm F1 so với checkpoint hợp nhất, nên checkpoint hợp nhất tiếp tục được dùng.

5.2.3 Lỗi kỹ thuật đã sửa trong quá trình huấn luyện

Lần đánh giá cuối epoch 1 trên bộ dữ liệu mở rộng từng dừng với lỗi OverflowError ("can't convert negative int to unsigned"). Nguyên nhân là chuỗi predictions do generate() sinh ra có thể chứa giá trị -100 giống labels, nhưng chỉ labels được lọc trước batch_decode. Cách sửa là thay -100 bằng pad_token_id cho cả hai ở compute_metrics và ở bước dự đoán test cuối. Ngoài ra, transformers phiên bản 5 trở lên không đọc được tokenizer ViT5 đã lưu nên ràng buộc được ghim ở dưới 4.58.

CHƯƠNG 6: ĐÁNH GIÁ THỰC NGHIỆM VÀ KẾT QUẢ

6.1 PHƯƠNG PHÁP VÀ CHỈ SỐ ĐO LƯỜNG

Báo cáo tách riêng ba mức đo để không lẫn lỗi của khâu này với khâu khác:

- **Retrieval:** Recall@k (đoạn đúng nằm trong k kết quả đầu) và MRR@k, với đoạn đúng xác định bằng đoạn gốc của câu hỏi sau chuẩn hóa NFC và khoảng trắng; không dùng nhãn đáp án A-D.
- **Reader:** Exact Match (EM) và token F1 sau chuẩn hóa chữ thường, dấu câu, khoảng trắng, khi **đưa đúng đoạn văn** cho ViT5.
- **Toàn chuỗi:** EM và F1 khi đoạn văn do bộ truy xuất chọn, đo bằng scripts/eval_e2e.py trên các câu hỏi có đoạn gốc nằm trong kho.

Cần lưu ý rằng EM/F1 của mô hình sinh đáp án từ ViMMRC không tương đương độ chính xác trắc nghiệm: mô hình không nhận bốn phương án mà phải tự sinh văn bản. Số liệu trên bộ hợp nhất cũng không so sánh trực tiếp được với số liệu trên ViMMRC thuần vì dev/test nay gồm hai nguồn.

6.2 KẾT QUẢ TRUY XUẤT

Bảng 6.1: Kết quả truy xuất đoạn văn

Phép đo	Số câu	Recall@1	Recall@5	Ghi chú
ViMMRC dev, kho 70 đoạn dev, BM25 cũ	564	71,45%	83,69%	MRR@5 0,7624
ViMMRC dev, kho 70 đoạn dev, cấu hình chọn	564	77,48%	86,52%	MRR@5 0,8123
Dev hợp nhất, kho 5.879 đoạn, hai tầng BM25	-	70,7%	85,1%	Trước reranker
Test hợp nhất, kho 5.879 đoạn	2.545	71,1%	85,9%	Trước reranker

Nguồn: Tác giả đo từ lexical_benchmark.json và STATUS.md (2026). Dòng 1-2 là kho đóng (đoạn đích có sẵn), chỉ phản ánh khả năng xếp hạng

Hai dòng đầu là đánh giá trên **kho đóng**: bài đọc đích đã nằm trong tập ứng viên nên không nên diễn giải là khả năng truy xuất tài liệu chưa thấy. Khi kho mở rộng từ 552 đoạn ViMMRC sang 5.879 đoạn, Recall@1 trên câu hỏi ViMMRC giảm từ 59,41% xuống 52,48% (404 câu), do kho lớn hơn có nhiều đoạn gây nhiễu; trong khi trên 2.166 câu ViQuAD Recall@1 đạt 74,52%. Thêm tầng Wikipedia với tỷ lệ 0,5 gần như không đổi (51,98% và 74,24%).

6.3 KẾT QUẢ ĐỌC HIỂU

Bảng 6.2: Kết quả mô hình đọc hiểu khi đưa đúng đoạn văn

Checkpoint	Test EM	Test token F1	Ghi chú
vit5-combined	41,05%	66,02%	Test hỗn hợp 2.716 câu
vit5-combined-cont	41,05%	66,55%	Đang dùng trong Docker
vit5-vimmrc-ft	9,89%	27,51%	Thước đo riêng, không so trực tiếp

Nguồn: Tác giả đọc từ test_metrics.json của ba checkpoint (2026)

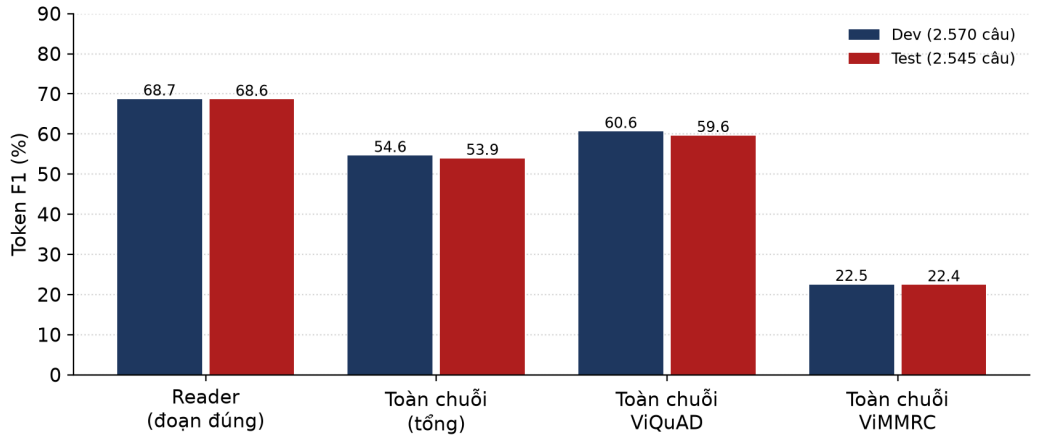
Mô hình đạt F1 66,55% nhưng EM chỉ 41,05%: khoảng cách này do đáp án tham chiếu thường là cụm dài, trong khi mô hình sinh diễn đạt khác ký tự dù đúng ý. Vì vậy F1 phản ánh chất lượng thực tế tốt hơn EM trên bài toán này.

6.4 KẾT QUẢ TOÀN CHUỖI VÀ HIỆU QUẢ XẾP HẠNG LẠI

Bảng 6.3: Kết quả toàn chuỗi truy xuất rồi đọc hiểu

Phép đo	EM	F1	F1 ViMMRC	F1 ViQuAD	F1 đoạn đúng
Dev, vit5-combined-cont (2.570 câu)	33,4	54,6	22,5	60,6	68,7
Dev, vit5-vimmrc-ft (2.570 câu)	32,4	54,4	23,9	60,1	68,1
Test, vit5-combined-cont (2.545 câu)	32,3	53,9	22,4	59,6	68,6

Nguồn: Tác giả đo bằng scripts/eval_e2e.py, báo cáo reports/e2e_*_kb_*.json (2026)

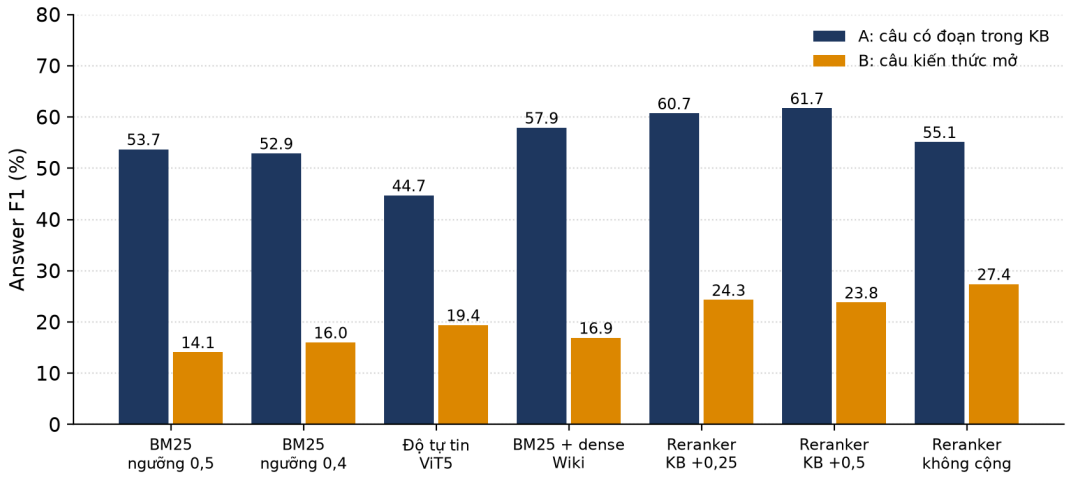


Hình 6.1: So sánh token F1 của khâu đọc hiểu (đoạn đúng) và toàn chuỗi trên tập dev và test

Nguồn: Tác giả tổng hợp từ reports/e2e_dev_kb_vit5-combined-cont.json và e2e_test_kb_vit5-combined-cont.json (2026)

Toàn chuỗi mất khoảng 14 điểm F1 so với đọc trên đoạn đúng (68,6 xuống 53,9 ở tập test), cho thấy truy xuất sai đoạn là nguồn lỗi lớn. Câu hỏi ViMMRC chỉ đạt F1 22,4 so với ViQuAD 59,6, vì văn bản sách giáo khoa và dạng câu hỏi trắc nghiệm khó trả lời bằng đoạn trích hơn.

Hình 6.2 so sánh các cách chọn tầng kho tri thức, đo trên 500 câu mỗi nhóm: nhóm A là câu có đoạn trong kho chọn lọc, nhóm B là câu kiến thức mở (đoạn ViQuAD test đã gỡ khỏi kho). Xếp hạng lại với thưởng 0,25 nâng nhóm A lên 60,7 và nhóm B lên 24,3 so với 52,9 và 16,0 của luật ngưỡng BM25 0,4. Bỏ thưởng làm nhóm B tăng lên 27,4 nhưng nhóm A tụt xuống 55,1; thưởng 0,5 cho A cao hơn (61,7) nhưng B thấp hơn (23,8). Mức 0,25 là thỏa hiệp, được chọn trên chính 400 câu đo nên hơi lạc quan.



Hình 6.2: Answer F1 của các cách chọn tầng kho tri thức (A: câu có đoạn trong kho, B: câu kiến thức mở)

Nguồn: Tác giả tổng hợp từ reports/routing_e2e.json, dense_routing_e2e.json, rerank_routing_e2e.json (2026)

6.5 KẾT QUẢ THỬ NGHIỆM VỚI MÔ HÌNH NGÔN NGỮ LỚN

Mục tiêu ban đầu là đạt 80-90% EM. Tác giả thử hai hướng dùng LLM chạy cục bộ qua Ollama (llm_eval.py): để LLM chấm tương đương về ý nghĩa với đáp án của ViT5, và để LLM 9 tỷ tham số tự sinh đáp án thay ViT5.

Bảng 6.4: Thử nghiệm dùng LLM để nâng độ chính xác

Hướng	Số câu	LLM-judge	EM ký tự	Token F1
ViT5 fine-tune, LLM chấm theo ý nghĩa	200	61,0%	43,5%	65,6%
ViT5 fine-tune, LLM chấm theo ý nghĩa	50	50,0%	44,0%	63,0%

Hướng	Số câu	LLM-judge	EM ký tự	Token F1
LLM 9B sinh trực tiếp, không dùng ViT5	50	54,0%	32,0%	61,7%

Nguồn: Tác giả đo bằng mrc-model_Huy/llm_eval.py (2026); hai dòng 50 câu cùng seed

Không hướng nào tiến gần 80-90%. Với 50 câu cùng seed, ViT5 và LLM 9B tương đương nhau (chênh vài điểm, nằm trong sai số của mẫu nhỏ), và 16/50 câu cả hai đều trả lời sai. LLM chấm theo ý nghĩa luôn cao hơn EM ký tự khoảng 10-20 điểm. Kết luận là thước đo EM ký tự bị chặn trên bởi cách diễn đạt đáp án, và việc cải thiện thật sự cần mô hình lớn hơn nhiều hoặc đổi thước đo, chứ không phải tinh chỉnh thêm ViT5.

6.6 KẾT QUẢ TẦNG HỘI THOẠI VÀ ĐỘ TRỄ

Bảng 6.5: Kết quả tầng hội thoại trên bộ câu hỏi tự viết (offline, kho mẫu 75 đoạn, bộ đọc heuristic)

Bộ đánh giá	Số mục	Lần chạy đầu	Sau khi sửa lỗi chung
Phát triển	157	130/157 = 82,8%	156/157 = 99,4% (đã chỉnh trên chính bộ này)
Held-out (viết sau khi chỉnh)	72	59/72 = 81,9%	70/72 = 97,2%

Nguồn: Tác giả đo bằng dialogue-manager_Huy/eval_dialogue.py; reports/dialogue_heldout_first_run.json (2026)

Con số trung thực nhất là 81,9% ở lần chạy đầu trên bộ held-out; con số 97-99% sau khi sửa đã được chỉnh theo chính các lỗi lộ ra nên mang tính lạc quan. Các phép đo này dùng kho mẫu và bộ đọc heuristic, chưa phải kho thật với ViT5.

Về độ trễ, trên Docker CPU 4 luồng (OMP_NUM_THREADS=4), truy xuất kèm xếp hạng lại mất khoảng 6 giây, ViT5 khoảng 1,4 giây; cùng STT (~3 giây) và Matcha-TTS (~1,3 giây) tổng cộng khoảng 12 giây mỗi câu. Tắt reranker (KB_RERANKER để trống) thì truy xuất còn 0,1-1,7 giây, tổng khoảng 6 giây, đổi lại độ chính xác thấp hơn. Bộ 110 câu ngoài tập dữ liệu và 35 câu khó đạt tỷ lệ trả lời hoàn chỉnh 100%, nhưng đây là tỷ lệ câu có cấu trúc ngữ pháp và dấu câu hợp lệ cho TTS, **không phải độ chính xác**.

CHƯƠNG 7: KHÓ KHĂN THỰC TẾ VÀ GIẢI PHÁP KHẮC PHỤC

7.1 TỔNG HỢP CÁC THÁCH THỨC KỸ THUẬT VÀ GIẢI PHÁP

Trong quá trình xây dựng, tác giả gặp các vấn đề được tổng hợp tại Bảng 7.1.

Bảng 7.1: Khó khăn kỹ thuật thực tế và giải pháp khắc phục

Khó khăn	Giải pháp
Dữ liệu ViMMRC nhỏ (5.273 câu), mô hình chỉ học một dạng câu hỏi	Gộp UIT-ViQuAD2.0, chia lại theo đoạn văn để tránh rò rỉ; 27.164 câu.
Chỉ mục bigram cho Wikipedia không vừa bộ nhớ máy xách tay	Tầng Wikipedia dùng bm25s không bigram, bù bằng truy xuất vector và reranker.
Điểm BM25 hai tầng không so sánh được trực tiếp	Ban đầu dùng tỷ lệ 0,4; sau đó để reranker chấm chung kèm thưởng 0,25.
Thưởng cho kho chọn lọc làm đoạn sai thắng đoạn đúng của Wikipedia	Chỉ thưởng khi điểm thô của reranker > -2.
Lỗi OverflowError khi đánh giá cuối epoch do giá trị -100	Thay -100 bằng pad_token_id cho cả predictions và labels.
transformers 5.x không đọc được tokenizer ViT5 đã lưu	Ghim phiên bản < 4.58; đặt TRANSFORMERS_NO_TF=1 để tránh treo khi import.
Train thêm cùng cấu hình làm dev loss tăng	Dừng ở checkpoint tốt nhất theo EM dev; không train thêm.
Muốn đạt 80-90% EM nhưng LLM 9B không hơn ViT5	Kết luận là giới hạn của bài toán và thước đo; báo cáo F1 là thước đo chính.
Reranker chậm (~4,5 giây mỗi câu trên CPU)	Cho phép tắt bằng KB_RERANKER; mặc định giữ bật để ưu tiên độ chính xác.

Nguồn: Tác giả đúc kết từ quá trình phát triển và thực nghiệm (2026)

CHƯƠNG 8: KẾT LUẬN VÀ ĐỀ NGHỊ

8.1 KẾT LUẬN

Phần việc đã hoàn thành chuỗi truy xuất, đọc hiểu và hội thoại cho hệ thống hỏi đáp giọng nói tiếng Việt, chạy được trên CPU. Truy xuất BM25 tự cài đặt tăng Recall@1 từ 71,45% lên 77,48% trên dev nhờ chọn cấu hình bằng dữ liệu huấn luyện; kho hai tầng với bộ xếp hạng lại nâng F1 câu hỏi trong kho từ 52,9 lên 60,7 và câu kiến thức mở từ 16,0 lên 24,3. Mô hình ViT5 đạt EM 41,05% và token F1 66,55% trên tập test hỗn hợp, toàn chuỗi đạt F1 53,9.

Các kết quả âm cũng có giá trị: fine-tune riêng ViMMRC không cải thiện, LLM 9B không vượt ViT5, và train thêm cùng cấu hình gây quá khớp. Chúng cho thấy giới hạn hiện tại đến từ bản chất bài toán và dữ liệu hơn là từ việc chưa tinh chỉnh đủ.

8.2 HẠN CHẾ CỦA ĐỀ TÀI

- **Về câu hỏi kiến thức mở:** F1 chỉ 24,3 với câu hỏi mà đoạn đáp án không có trong kho chọn lọc; truy xuất sai đoạn là nguyên nhân chính.
- **Về dữ liệu ViMMRC:** F1 toàn chuỗi trên câu ViMMRC chỉ 22,4; mô hình sinh không nhận bốn phương án nên không so sánh được với độ chính xác trắc nghiệm của các công bố khác.
- **Về tầng hội thoại:** mới đo offline bằng kho mẫu và bộ đọc heuristic; một số lượt đại từ trong hội thoại dài (ví dụ "ông ấy") vẫn truy xuất sai đoạn. Chưa đo trên kho thật kèm ViT5.
- **Về độ tin cậy số liệu:** mức thưởng 0,25 được chọn trên chính bộ đo nên hơi lạc quan; các mẫu so sánh LLM chỉ 50-200 câu nên có sai số lớn; chưa có bộ phát hiện câu hỏi ngoài miền đã hiệu chỉnh.

8.3 ĐỀ NGHỊ VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

- **Cải thiện truy xuất:** tách từ tiếng Việt theo từ ghép trước khi lập chỉ mục BM25 và tinh chỉnh bộ mã hóa vector trên dữ liệu hỏi đáp của đề tài.
- **Nâng cấp mô hình đọc hiểu:** thử mô hình seq2seq lớn hơn hoặc LLM cỡ lớn khi có GPU mạnh hơn, và chấm bằng F1 hoặc đánh giá theo ý nghĩa thay vì EM ký tự.
- **Xử lý đoạn dài:** áp dụng chọn đoạn con theo câu hỏi (*question_chunks*) ở cả huấn luyện lẫn suy luận thay vì cắt đầu đoạn; thử nghiệm hiện có chưa cho thấy cải thiện ở suy luận.
- **Đánh giá hội thoại trên hệ thống thật:** đo tầng hội thoại với kho đầy đủ và ViT5 trên bộ hội thoại nhiều lượt có nhãn do người chấm.

TÀI LIỆU THAM KHẢO

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024). *BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*. arXiv:2402.03216.

Karpukhin, V., Oguz, B., Min, S., et al. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP 2020*, 6769-6781.

Lù, X. H. (2024). *BM25S: Orders of magnitude faster lexical search via eager sparse scoring*. arXiv:2407.03618.

Nogueira, R., and Cho, K. (2019). *Passage re-ranking with BERT*. arXiv:1901.04085.

Phan, L., Tran, H., Nguyen, H., and Trinh, T. H. (2022). ViT5: Pretrained text-to-text transformer for Vietnamese language generation. In *Proceedings of NAACL 2022 Student Research Workshop*.

Raffel, C., Shazeer, N., Roberts, A., et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1-67.

Robertson, S., and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

UIT NLP. (2023). *ViMMRC 2.0: Vietnamese multiple-choice machine reading comprehension*. Hugging Face Datasets, uitnlp/vimmrc2.0 (dữ liệu truy cập có kiểm soát).

Wang, L., Yang, N., Huang, X., et al. (2024). *Multilingual E5 text embeddings: A technical report*. arXiv:2402.05672.

taidng. (2023). *UIT-ViQuAD2.0*. Hugging Face Datasets, taidng/UIT-ViQuAD2.0.